NAME: MUHAMMAD MUSTAFA

REG#: FA22-BAI-026

Overview:

The provided code implements a genetic algorithm (GA) for solving the 2048 puzzle. The 2048 puzzle is a sliding puzzle where the player combines tiles with the same number to reach a tile with the number 2048. The genetic algorithm is used to evolve a population of individuals (game board configurations) to find a solution to the puzzle.

Genetic Algorithm for 2048 Puzzle Solver

Class: GA2048

1. **Constructor (__init__):**

```
def __init__(self, gridSize, populationSize):
    self.population = dict()
    self.gridSize = gridSize
    self.populationSize = populationSize
    self.totalFitness = 0

    i = 0
    while i < populationSize:
        individual = np.random.randint(0, 15, size=(gridSize, gridSize))
        self.population[i] = [individual, self.calculateFitness(individual)]
        self.totalFitness += self.population[i][1]
        i += 1
```

Explanation:

- This method is the constructor of the GA2048 class.
- It initializes the genetic algorithm with the specified gridSize and populationSize.
- Creates a dictionary population to store individuals and their fitness values.
- Initializes attributes gridSize, populationSize, and totalFitness.

```
def __init__(self, gridSize, populationSize):
    """
    Initializes the Genetic Algorithm for solving the 2048 puzzle.

    Parameters:
    - gridSize (int): Size of the square grid for each individual.
    - populationSize (int): Size of the population of individuals.

    Attributes:
    - population (dict): Dictionary containing individuals and their fitness values.
    - gridSize (int): Size of the individual grid.
    - populationSize (int): Size of the population.
    - totalFitness (int): Total fitness of the population.
    """
```

2. Method: ‘calculateFitness’

```
def calculateFitness(self, individual):
    # Sum up the values in the individual matrix as a simple fitness measure
    return np.sum(individual)
```

Explanation:

- Calculates and returns the fitness of a given individual.
- The fitness is calculated as the sum of all values in the individual matrix.

Parameters:

- **individual (numpy.ndarray):** Matrix representing an individual.

Returns:

- **int:** Fitness value of the individual (sum of matrix values).

3. Method: “updatePopulationFitness”

```
def updatePopulationFitness(self):
  self.totalFitness = sum(self.population[individual][1] for individual in self.population)
```

Explanation:

- Updates the **totalFitness** attribute by summing up the fitness values of all individuals in the population.

4. Method: “selectParents”

```python
def selectParents(self):
  rouletteWheel = []
  wheelSize = self.populationSize * 5
  h_n = [self.population[individual][1] for individual in self.population]

  for j in range(self.populationSize):
      individualLength = round(wheelSize * (h_n[j] / sum(h_n)))
      if individualLength > 0:
          rouletteWheel.extend([j] * individualLength)

  random.shuffle(rouletteWheel)
  parentIndices = [rouletteWheel[random.randint(0, len(rouletteWheel) - 1)] for _ in range(self.populationSize)]
  newGeneration = {i: self.population[parentIndices[i]].copy() for i in range(self.populationSize)}
  del self.population
  self.population = newGeneration.copy()
  self.updatePopulationFitness()
```

Explanation:

- Implements roulette wheel selection to choose parents based on their fitness.
- Creates a roulette wheel based on individual fitness values.
- Randomly selects parents from the wheel, updates the population, and updates the total fitness.

5. Method: “generateChildren”

```python
def generateChildren(self, crossoverProbability):
    numberOfPairs = round(crossoverProbability * self.populationSize / 2)
    individualIndices = list(range(self.populationSize))
    random.shuffle(individualIndices)

    i, j = 0, 0
    while i < numberOfPairs:
        crossoverPoint = random.randint(0, self.gridSize - 1)
        child1 = np.vstack((self.population[j][0][:crossoverPoint], self.population[j+1][0][crossoverPoint:]))
        child2 = np.vstack((self.population[j+1][0][:crossoverPoint], self.population[j][0][crossoverPoint:]))

        self.population[j] = [child1, self.calculateFitness(child1)]
        self.population[j+1] = [child2, self.calculateFitness(child2)]

        i += 1
        j += 2
```

Explanation:

- Generates children through crossover operation.
- Randomly selects pairs of individuals and performs crossover at a random point in their matrices.
- Updates the population with the created children and recalculates their fitness.

6. Method: “mutateChildren”

```python
def mutateChildren(self, mutationProbability):
  numberOfBits = round(mutationProbability * self.populationSize * self.gridSize * self.gridSize)
  totalIndices = list(range(self.populationSize * self.gridSize * self.gridSize))
  random.shuffle(totalIndices)
  swapLocations = random.sample(totalIndices, numberOfBits)

  for loc in swapLocations:
      individualIndex, remainder = divmod(loc, self.gridSize * self.gridSize)
      row, col = divmod(remainder, self.gridSize)
      self.population[individualIndex][0][row, col] = 1 - self.population[individualIndex][0][row, col]

  self.updatePopulationFitness()
```

Explanation:

- Mutates children by flipping a random bit in their matrices based on the specified mutation probability.
- Updates the population with the mutated children and recalculates their fitness.

7. Example:

```python
# Example:
gridSize, populationSize = 4, 10
i = 0
instance = GA2048(gridSize, populationSize)
# while i<=100:
while True:
    instance.selectParents()
    instance.generateChildren(0.8)
    instance.mutateChildren(0.03)

    print(instance.population)
    print(instance.totalFitness)
    print(i)

    i += 1
    found = any(instance.population[individual][1] == gridSize for individual in instance.population)

    if found:
        break
```

Explanation:

- Demonstrates the usage of the GA2048 class to solve the 2048 puzzle.
- Initializes an instance, iterates through the genetic algorithm operations, and prints relevant information.
- Breaks the loop if a solution is found.

Output Explanation:

1. instance.population:

- This prints the current population at each iteration.
- The population is a dictionary where keys are individual indices, and values are lists containing the individual matrix and its fitness.

Example:

```
{0: [array([[1, 1, 0, 0],
            [1, 1, 0, 0],
            [1, 1, 0, 0],
            [1, 1, 0, 0]]), 8],
 1: [array([[1, 1, 0, 0],
            [1, 1, 0, 0],
            [1, 1, 0, 0],
            [1, 1, 0, 0]]), 8],
 ...
}
```

2. **instance.totalFitness:**
   - This prints the total fitness of the current population at each iteration.

Example:

```
80
```

3. **Iteration Number:**
   - This prints the current iteration number.

Example:

```
1
```

4. **Break Condition:**
   - The loop breaks if a solution is found.
   - A solution is found if any individual in the population has a fitness equal to the gridSize (i.e., 4 in this case).

Summary:

- The output provides insight into the evolving population, total fitness, and the progression of iterations.
- The algorithm continues to iterate until a solution is found or a specified condition is met.
- The solution is a matrix whose sum of values equals the gridSize.